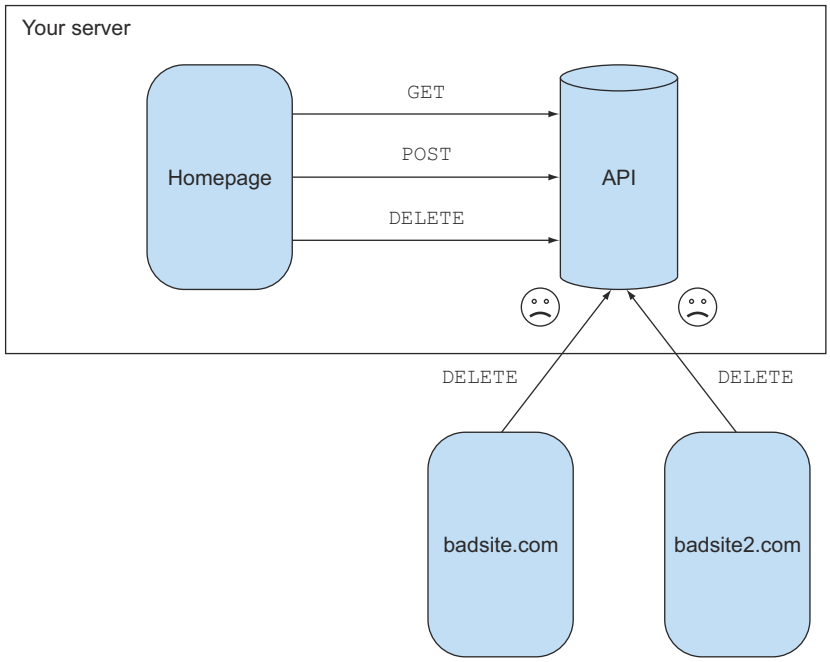


Figure 4.4 A CORS nightmare scenario. Cross-origin requests are made without any permission from the server. Thankfully this isn't how CORS is implemented.

preserved, because the request is never made unless the server grants permission. An existing server that knows nothing about CORS can safely ignore the preflight request, and the browser will not forward the actual request to the server, as shown in figure 4.5.

To return to the story, after learning about the CORS preflight request, you rest a little easier that night, knowing that your server won't receive any unauthorized requests from other people's servers.

This story demonstrates why the preflight was introduced: it allows cross-origin requests to be introduced to the web in a way that doesn't adversely affect existing servers. Now that you know why the preflight request exists, let's modify the sample code to trigger a preflight request.

4.2 Triggering a preflight request

Chapter 3 introduced a sample blogging app that loads blog posts using CORS. Let's modify that to let the user delete blog posts. The standard method for deleting data in a REST API is to use the HTTP `DELETE` method, so we'll use that here.

Listing 4.1 modifies client.html to display a Delete link next to each blog post. Clicking the link calls the `deletePost` JavaScript function. The `deletePost` function deletes a blog post by sending a `DELETE` request to the URL /api/posts/{ID}, where `{ID}` is the blog post's ID. For example, a request to /api/posts/1 would delete the blog post with an ID of 1. If the delete is successful, the post is removed from the page.